Reporte Profundo: Emparejamiento y Conectividad entre Dispositivos — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar cómo funciona el emparejamiento actual, cómo debería funcionar (frictionless), y qué estrategias de fallback existen para que los dispositivos se encuentren y comuniquen sin intervención del usuario.

---

1. Estado actual del emparejamiento (lo que YA existe)

1.1 Flujo de pareo por QR

```
COMPUTADOR CENTRAL (Rol A / Caja)
        │
        ├──► Genera QR cifrado (BO1./BOF.) con:
        │       - Identidad del hub (ECDH P-256)
        │       - URL de instalación (bonaorigen.app/instalar)
        │       - Token LAN (x-crozzo-lan-token)
        │       - locationId, deviceId, businessId
        │
        └──► Muestra QR en pantalla grande

CELULAR/TABLET (Rol B)
        │
        ├──► Escanea QR con cámara
        ├──► Descifra payload (ECDH + firma)
        ├──► Guarda credenciales en localStorage
        ├──► Se conecta al servidor LAN (IP:port del QR)
        └──► Comienza sync (HTTP poll + WS push)
```

Archivos clave:
- `app/core/CrozzoPairingSeal.js` — Generación/cifrado del QR (ECDH P-256, firma)
- `app/modules/CrozzoPairingQrReader.js` — Lectura del QR en tablet/celular
- `app/infra/CrozzoPairingAutoConnect.js` — Auto-conexión tras pareo
- `src-tauri/src/crozzo_mdns.rs` — Descubrimiento mDNS

1.2 Flujo mDNS (misma red, sin QR)

```
Caja (Rol A) inicia → mDNS anuncia _crozzo-pos._tcp.local.
        │
Tablet (Rol B) busca → encuentra caja → obtiene IP:port + token
        │
Tablet se autentica → sync comienza automáticamente
```

Ventaja: Cero fricción si ambos están en la misma WiFi.

1.3 Flujo Cloud (internet disponible)

```
Dispositivo A → Supabase (push) → Dispositivo B (pull)
        │
        └──► Funciona cross-network (diferentes redes)
        └──► Fallback automático si LAN falla
```

1.4 Flujo Offline (sin red)

```
Dispositivo A → datos locales (SQLite/localStorage)
        │
        └──→ Sync cuando vuelve conexión
        └──→ Contingencia fiscal (48h SLA)
```

---

2. El flujo ideal (lo que el usuario quiere)

```
1. Usuario instala app en computador central (Rol A)
   → Configura impresoras, base de datos, empresa
   → App genera QR de pareo automáticamente

2. Usuario instala app en celular (Rol B)
   → App pide permisos (red, cámara, Bluetooth) UNA vez
   → Usuario escanea QR del computador
   → Celular se conecta automáticamente

3. A partir de ahí:
   → Caja sabe que hay un celular "X" unido
   → Celular sabe que hay una caja "Y" y otros dispositivos
   → Todos se comunican automáticamente:
     - Misma red WiFi → mDNS + HTTP/WS (más rápido)
     - Hay internet → Cloud sync (Supabase)
     - Sin red → modo offline (datos locales)
   → Si un dispositivo se cae → los demás siguen funcionando
   → Si vuelve → sync automático

4. El usuario NO hace nada más. Cero configuración manual.
```

Clave: El usuario solo escanea un QR una vez. Después, todo es automático.

---

3. Estrategias de fallback (qué pasa si falla algo)

3.1 Escenario: misma red WiFi, sin internet

```
Caja y tablet en misma WiFi → mDNS descubre caja
        │
        ├──→ HTTP polling (port 3000) → sync de comandas
        ├──→ WebSocket push (port 3001) → tiempo real
        └──→ NO necesita internet → funciona offline
```

Resultado: ✅ Funciona. Los dispositivos se comunican por LAN.

3.2 Escenario: sin WiFi, solo Bluetooth

```
Caja crea hotspot WiFi (si tiene capacidad)
        │
        └──→ Tablet se conecta al hotspot → mDNS funciona
             └──→ Sync por LAN

Si NO hay hotspot:
```

```
        │
        └──► Bluetooth LE (si ambos soportan)
             └──► Requiere tauri-plugin-blec
             └──► Alcance ~10-30m, ancho de banda bajo
             └──► Solo para datos pequeños (comandas, no archivos)
```

Resultado: ⚠️ Parcialmente viable. Bluetooth LE funciona para datos pequeños pero requiere plugin nativo y permisos.

3.3 Escenario: sin red de ningún tipo

```
Todos los dispositivos en modo offline
        │
        ├──► Datos locales (SQLite en Tauri, localStorage en web)
        ├──► Contingencia fiscal (48h SLA para timbrado)
        └──► Sync cuando vuelve conexión
```

Resultado: ✅ Funciona en modo offline. Al volver la red, sync automático.

3.4 Escenario: un dispositivo se cae

```
Dispositivo A cae → Dispositivos B, C siguen funcionando
        │
        ├──► B y C siguen sincronizando entre sí (LAN o cloud)
        ├──► Cuando A vuelve → sync automático (pull de cambios)
        └──► No se pierde información (persistencia local)
```

Resultado: ✅ Resiliente. Cada dispositivo es independiente.

3.5 Escenario: WiFi con internet

```
Todos los dispositivos → Supabase (cloud)
        │
        ├──► Sync bidireccional
        ├──► Funciona cross-network (diferentes redes)
        └──► Latencia mayor que LAN
```

Resultado: ✅ Funciona. Fallback automático si LAN falla.

---

4. Análisis de viabilidad por tecnología

4.1 mDNS (ya implementado)

Aspecto	Detalle
Protocolo	Multicast DNS (RFC 6762) + DNS-SD (RFC 6763)
Puerto	UDP 5353
Alcance	Misma subred (LAN)
Fricción	Cero (automático)
Confiabilidad	Alta (si multicast no está bloqueado)
Estado en Crozzo	✅ Implementado en Rust (`crozzo_mdns.rs`)
Riesgo	Firewalls/routers pueden bloquear multicast (mDNS)

4.2 HTTP Polling + WebSocket (ya implementado)

Aspecto	Detalle
Protocolo	HTTP (port 3000) + WebSocket (port 3001)
Alcance	Misma red LAN
Fricción	Cero (automático tras mDNS)
Confiabilidad	Alta
Estado en Crozzo	✅ Implementado en Rust
Riesgo	WS broadcast panic (bug latente)

4.3 WebRTC Data Channels (investigado, no implementado)

Aspecto	Detalle
Protocolo	WebRTC (ICE + STUN/TURN + SCTP)
Alcance	LAN e internet (cross-network)
Fricción	Requiere signaling inicial (QR o cloud)
Confiabilidad	Media (NAT traversal puede fallar)
Estado en Crozzo	❌ No implementado
Ventaja	P2P directo, baja latencia, no satura DB
Desventaja	Complejidad de signaling + ICE
Viabilidad en Tauri	✅ WebRTC disponible en webview (Chromium/WebView2)
Recomendación	✅ Complemento para transferencia de archivos grandes

4.4 Bluetooth LE (investigado, no implementado)

Aspecto	Detalle
Protocolo	Bluetooth Low Energy (GATT)
Alcance	~10-30 metros
Fricción	Requiere permisos + pairing
Confiabilidad	Media (interferencia)
Estado en Crozzo	❌ No implementado
Ventaja	No requiere WiFi ni internet
Desventaja	Ancho de banda bajo, plugin nativo requerido
Viabilidad en Tauri	⚠️ Requiere `tauri-plugin-blec`
Recomendación	⚠️ Solo para impresoras/dispositivos cercanos

4.5 WiFi Hotspot (investigado, no implementado)

Aspecto	Detalle
Protocolo	WiFi AP mode
Alcance	~30-100 metros
Fricción	Requiere que un dispositivo cree hotspot
Confiabilidad	Media
Estado en Crozzo	❌ No implementado
Ventaja	No requiere router
Desventaja	Complejidad de gestión, no estándar en web
Viabilidad en Tauri	❌ Requiere APIs nativas de OS
Recomendación	❌ No recomendado (mDNS sobre WiFi existente es suficiente)

4.6 Transferencia por link (estilo wormhole/croc)

Aspecto	Detalle
Protocolo	HTTP/WebSocket relay
Alcance	Cross-network (internet)
Fricción	Compartir link/código

Confiabilidad	Alta (relay server)
Estado en Crozzo	❌ No implementado
Ventaja	Frictionless, funciona cross-network
Desventaja	Requiere relay server, solo archivos
Viabilidad en Tauri	✅ Se puede implementar con endpoint HTTP
Recomendación	✅ Útil para exportar reportes/backups

---

5. Permisos requeridos por plataforma

5.1 Android

```xml
<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.CHANGE_WIFI_MULTICAST_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  <!-- para mDNS/BLE -->
```

Runtime: Android 6+ requiere request de `ACCESS_FINE_LOCATION` en runtime.

5.2 iOS

```xml
<!-- Info.plist -->
<key>NSLocalNetworkUsageDescription</key>
<string>Crozzo necesita acceso a la red local para sincronizar con otros dispositivos</string>
<key>NSBonjourServices</key>
<array>
  <string>_crozzo-pos._tcp</string>
</array>
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Crozzo necesita Bluetooth para conectar con dispositivos cercanos</string>
```

Runtime: iOS 14+ muestra prompt de red local en primer uso.

5.3 Windows (Tauri)

- **Firewall**: Windows puede bloquear el puerto 3000. Se necesita prompt en primer uso.
- **No requiere permisos especiales** para mDNS/HTTP en LAN.

5.4 macOS

- **Local network permission**: System prompt en primer uso (macOS 11+).
- **No requiere permisos especiales** para HTTP en LAN.

---

6. Flujo de emparejamiento ideal (paso a paso)

Paso 1: Instalación en computador central
```
1. Usuario instala Crozzo en computador (Rol A / Caja)
2. Configura empresa, impresoras, base de datos
3. App genera QR de pareo automáticamente (ECDH P-256 + firma)
```

4. QR se muestra en pantalla grande
```

### Paso 2: Instalación en celular/tablet
```
1. Usuario instala APK en celular (Rol B)
2. App pide permisos (red, cámara, Bluetooth) — UNA vez
3. Usuario escanea QR del computador
4. Celular descifra payload → guarda credenciales
5. Celular se conecta al servidor LAN (IP:port del QR)
6. Sync comienza automáticamente
```

### Paso 3: Descubrimiento automático de dispositivos
```
1. Caja anuncia _crozzo-pos._tcp por mDNS
2. Tablet en misma red descubre caja automáticamente
3. Tablet se autentica con token
4. Caja registra: "Tablet X se unió"
5. Caja notifica a otros dispositivos: "Tablet X está activa"
6. Todos los dispositivos actualizan su lista de peers
```

### Paso 4: Comunicación continua
```
Prioridad de conexión (automática):
  1. ¿Misma red LAN? → mDNS + HTTP/WS (más rápido)
  2. ¿Hay internet? → Cloud sync (Supabase)
  3. ¿LAN + internet? → Ambos en paralelo (redundancia)
  4. ¿Sin red? → Offline (datos locales)

Si un dispositivo se cae:
  → Los demás siguen funcionando
  → Cuando vuelve → sync automático
  → No se pierde información (persistencia local)
```

### Paso 5: Fallback si no hay red
```
1. Todos los dispositivos en modo offline
2. Datos locales (SQLite/localStorage)
3. Contingencia fiscal (48h SLA para timbrado)
4. Sync cuando vuelve conexión
```

---

## 7. Bugs latentes que hay que corregir

| # | Bug | Archivo | Fix propuesto | Prioridad |
|---|-----|---------|---------------|-----------|
| 1 | Digest LAN ciego | `CrozzoLanOpsSync.js` | Incluir estado en hash (no solo IDs+count) | 🔴 Alta |
| 2 | WS broadcast panic | `crozzo_lan_ws.rs` | Reemplazar `Handle::block_on` por async await | 🔴 Alta |
| 3 | Body 64KB truncation | `crozzo_lan_sync_server.rs` | Leer en loop hasta EOF | 🟡 Media |
| 4 | Outbox sin idempotencia | `CrozzoComandasCloudSync.js` | Agregar consecutivo/idempotency key | 🟡 Media |
| 5 | Multi-writer carrito | `CrozzoPosRuntimeCloud.js` | Lock optimista con version field | 🟡 Media |
```

---

8. Roadmap de implementación

Fase A — Corregir bugs críticos (1-2 días)
- Fix digest LAN (incluir estado en hash)
- Fix WS panic (async await en lugar de block_on)
- Fix body truncation (read loop)
- Agregar `since` param a /api/comandas

Fase B — Frictionless pairing (3-5 días)
- Verificar permisos en Android/iOS (manifest + runtime)
- Auto-discovery por mDNS (ya implementado, verificar en campo)
- QR de pareo como fallback (ya implementado)
- Cloud fallback automático (ya implementado)
- Guía de configuración de permisos en la app

Fase C — WebRTC P2P (1-2 semanas)
- Implementar signaling vía Supabase (ya existe infra)
- Data channel para sync de datos (no solo archivos)
- Fallback a HTTP/WS si WebRTC falla
- Transferencia de archivos grandes (reportes, backups)

Fase D — Transferencia por link (1 semana)
- Endpoint para generar link de descarga
- Descarga por código/link (estilo wormhole)
- Útil para exportar reportes sin saturar DB

---

9. Riesgos y limitaciones

Riesgo	Mitigación
mDNS bloqueado por firewall/router	Fallback a cloud sync
WebRTC NAT traversal falla	TURN relay (costo de servidor)
Bluetooth alcance limitado	Solo para impresoras/dispositivos cercanos
Permisos denegados por usuario	Guía de configuración en app
Red intermitente	Modo offline + sync al volver
Saturación de base de datos	WebRTC P2P para datos grandes
Dispositivo se cae	Persistencia local + sync al volver

---

10. Conclusión

El sistema **ya tiene conectividad LAN real e implementada** (HTTP + WS + mDNS + auth + persistencia + cloud). No es "inexistente" ni "pendiente". Los problemas son bugs latentes específicos.

Para lograr el objetivo de "sin fricción":
1. **Corregir los 2 bugs críticos** (digest ciego + WS panic) → 1-2 días
2. **Verificar permisos** en Android/iOS → 1 día
3. **Implementar WebRTC P2P** como complemento → 1-2 semanas
4. **Transferencia por link** para archivos grandes → 1 semana

El resultado final: un celular se conecta a la caja automáticamente por mDNS (misma red) o por cloud (internet), sin configuración manual, con fallback a offline si no hay red. El usuario solo escanea un QR una vez. Después, todo es automático.